

A Repository-Based System for Research and Publication

Architecture, evidence and iteration in a Lean research repository

WILL COOK*

6 October 2026

ABSTRACT

A research repository can preserve a proof while losing the conditions under which its result was obtained. This paper describes a public Lean research repository that keeps questions, sources, proof attempts, papers and review decisions together. Python tools retrieve selected records, save proof checks and rebuild documentation. Two publication records serve different purposes: one connects a paper statement to registered formal supports; the other connects an explanatory passage to source excerpts and an authored justification. Neither automatically checks that the explanation follows from its sources. Researchers choose the work, and maintainers decide which changes to accept.

In a historical author-run trial, publication checks rejected nine of ten deliberately false edits and accepted a false claim that an unresolved step was complete. We explain that failure, the limited follow-up, and the work still needed to assess whether the repository helps unfamiliar researchers.

1 Introduction

When research resumes after a change of contributor, session or draft, the next researcher must recover more than the latest answer. They need to know which question was being asked, which assumptions a result needs, why an earlier approach failed, and where those facts were recorded. A repository can contain all the relevant files and still leave this reconstruction to its reader. The system described here makes selected relationships explicit and gives the reader a route from a short account to the sources behind it.

Consider a revision to a paper about Erdős Problem 257. The unrestricted question asks whether every infinite subseries of a particular reciprocal series is irrational. The repository contains a theorem that answers this under a condition on the selected exponents. An editor could leave the theorem and its Lean proofs untouched while shortening its introduction to say that the problem had been solved. Every formal proof could remain valid, although the new explanation would be false. The condition, stated in Appendix B, is what separates the proved result from the open question [1, 2].

The repository addresses this problem with two records. A paper-to-proof record identifies the formal results registered as support for a particular statement. A passage-to-source record identifies the evidence for an explanation and preserves the author's reason for citing it. Checks detect changes to the recorded statements, files and source excerpts; a reviewer must decide whether the explanation is justified. The surrounding workflow saves attempts and review decisions so that a later contributor can inspect both accepted work and recorded failures.

The implementation discussed here is the public Erdős research repository, organised around 8 problems. A separate verification repository holds selected formal challenges and comparison

* *Authorship and AI use.* Will Cook built and directed the research infrastructure and reviewed the claims when he could. AI agents did most of the research and drafting. Cook did not independently verify every claim.

records. The public reading pages present material drawn from the research repository. A larger private working environment is outside this public implementation; a public clone does not require it. The account below concerns the supplied source snapshot, not the current state of a running service.

2 Starting a task and keeping its working record

2.1 Choose the question, then retrieve its context

A researcher or agent chooses a task: investigate a question, check an argument, inspect prior work, or revise an explanation. The entry script uses that request to recommend instructions, source files and commands. It is a routing aid. It neither selects the research programme nor runs the whole task. For the illustrative revision, the useful starting request is to find the existing Problem 257 result and its unresolved boundary, before editing the introduction.

The lookup tools connect a problem to its reviewed claims, paper locations and declaration names. From there, the contributor can read the source paper or inspect a relevant Lean declaration and its dependencies. The retrieved record gives the contributor a starting point: read the claim, follow its paper location, then inspect its formal support. A missing route or an incomplete record leaves work to do, rather than establishing that no relevant result exists.

The files have different jobs. Lean source contains the formal definitions and proofs. The reviewed claim register records selected public assertions and their unresolved obligations. Paper sources contain the argument and explanation. Indexes and reading pages help find these materials, and are rebuilt from their source files. Editing a generated page directly would therefore leave the underlying record unchanged and risk losing the edit on the next rebuild.

There is no single complete graph of the research. The declaration inventory covers the imported Lean environment, while exact dependency indexes are built for selected roots. The broader interpretation index also includes descriptions inferred from context. A displayed connection can consequently mean a formal dependency, a reviewed support relation or a navigation suggestion. Its type and source matter. Adding a Lean module does not automatically add a reviewed public claim.

2.2 Check an attempt and save what happened

For a proof task, the optional workbench sends a candidate Lean snippet to `lake env lean --stdin --json`. It saves the input, a content hash, the environment information and the last twelve lines of returned diagnostics. A probe is recorded as accepted only when Lean exits successfully, reports no errors and leaves no sorry. A failed proof, an unfinished proof and a timeout have different recorded outcomes. The next session can inspect the saved attempt instead of relying on a prose recollection of success.

A contributor may then attach a claim to an accepted probe. The workbench verifies that the referenced probe exists and that its saved source still matches its hash. This protects the link to the attempt that was checked. It does not establish that the contributor's description is entailed by that source.

A useful failed attempt records the question it addressed, the code or argument tried, the observed failure and the next unresolved step. A later contributor can use this account to decide whether to revisit the attempt or try a different approach. Contributors can also work through ordinary issues and pull requests without a workbench session. The journal consequently records only the work entered into it.

3 Statements, explanations and their evidence

The Problem 257 revision reaches a different kind of task at publication. A valid proof answers the formal statement presented to Lean [11]. A paper also says what the result means, how it relates to the original question and what it owes to previous work. Those claims can change without changing the proof. The publication records separate these objects so that a reviewer can see which check bears on which part of the paper.

3.1 A statement and its registered formal supports

The paper-to-proof record starts with an occurrence of a statement in a paper: its label, source location and text identity. It names the Lean declarations registered as support, at specified revisions. The record distinguishes exact support, support obtained by specialising a stronger result, support that depends on a named input, and a statement with no registered Lean support. The same mathematical result may have several paper occurrences, each with its own record.

When a paper statement specialises stronger formal results, the author supplies a relation note explaining the connection to its registered set of supports. The resolver checks that this note was written for the current paper statement and the recorded Lean statements. If either changes, it requires review of the note. The note explains the relation of the paper occurrence to the whole registered set; it is attached to the coverage row rather than separately to each declaration. The supports are selected entries, not the full transitive dependencies of their proofs.

Selected statements also have records from the separate verification repository. The resolver checks the named revision, entry, run and saved outcome, including the recorded Lean and independent-kernel acceptance and permitted axioms. Comparator checks a formal result against a separately stated formal challenge [19, 20]. Their scope is the particular statements and definitions compared; they do not establish that an informal problem was formalised as its author intended.

3.2 An explanation and its cited sources

The second record binds an explanatory passage to evidence. Each entry contains the passage text, source paths and line ranges, hashes of the source files and excerpts, an evidence class, and the author's justification for using those sources. An entry may cover several sentences or a displayed formula. The evidence class distinguishes implemented behaviour, a historical report, a proposed method and literature discussion.

The checker verifies that every marked passage has a corresponding entry, that the recorded hashes match, and that a justification is present. It does not decide whether that justification is sound. An author could update the hashes and write a false justification for the unrestricted Problem 257 claim, and these checks alone would accept the binding.

3.3 What the editor must still decide

For the illustrative revision, the editor should compare three things: the original open question, the theorem's hypotheses and conclusion, and the proposed introduction. The weighted condition restricts which exponent sets the theorem covers. The conclusion applies to every infinite subset of such a set and every integer base at least two. A faithful short introduction can omit the formula while retaining that restriction. It cannot turn a sufficient condition into an answer for all exponent sets.

The same review must retain the relation to earlier mathematics. Erdős's reciprocal-summability criterion and Duverney and Tachiya's selection and averaging arguments are relevant antecedents [3, 4, 2]. The owning paper distinguishes their contributions from its weighted

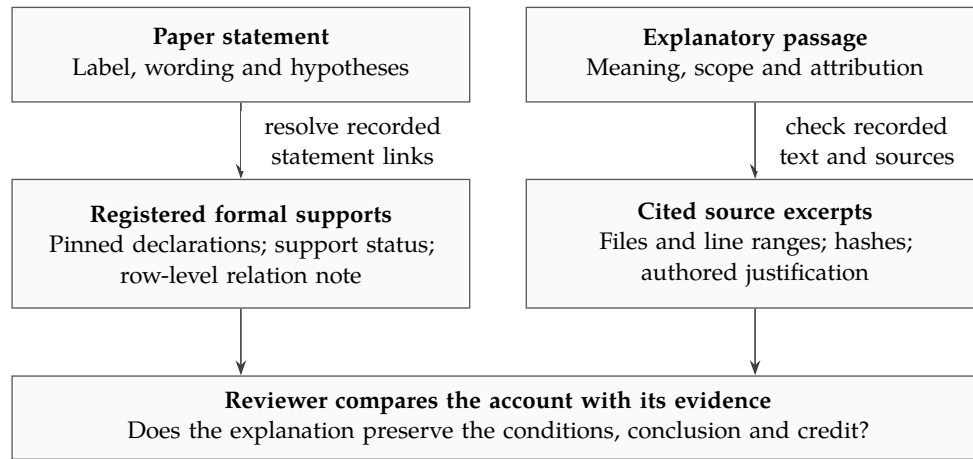


Figure 1: Two records, with different checking tasks. Arrows show which information is inspected. They do not assert logical implication or automatic approval. Formal acceptance does not settle whether the explanatory passage is faithful to the result.

condition and additional averaging step. A list of proof links would leave that distinction unexplained. Appendix B gives the precise scope and credit without repeating the specialist derivation.

4 Revising, reviewing and integrating a change

4.1 Prepare a draft without changing its evidence

The paper tools prepare material for an author or editor. They collect a result's statement, supports, attribution and unresolved obligations into a bounded dossier. The short-paper tool audits labels, references, citations and coverage bindings, and can render a draft from that dossier. A separate assembler joins authored sections while retaining their references. These tools operate on supplied files without calling a language model. A person or external model drafts the prose, and the reviewer judges it.

An editor can work from a frozen packet rather than a live checkout. The packet records the exact input bytes, the files that may be changed and the expected return format. On return, the checker compares the input hashes and permitted paths, checks the required accounting, and runs the native manuscript checks on an isolated copy. A stale input or an unauthorised path is grounds for rejection. The checker uses its own native validation code, without running scripts from the return. It produces findings for the maintainer, who decides whether to apply the edit.

For the Problem 257 edit, the return would contain the revised introduction and its updated passage-to-source entry. The introduction would describe a sufficient weighted condition for irrationality, while the theorem and its formal supports remained unchanged. The reviewer would compare the revision with the theorem and the earlier sources before accepting it. This is an example of the review procedure, not an additional recorded trial.

Permission to edit the live repository is separate from the kind of task being done. An advisory contributor can propose a proof correction, literature note or rewrite; an integrating maintainer can accept and apply it. The project calls these access roles Type B and Type A. They describe authority over files, not mathematical competence or independence. Ordinary issues and pull requests remain valid contribution routes, with maintainers responsible for deciding what belongs in the repository.

4.2 Record a decision and rebuild the affected material

For contributions entered through the exchange, the research journal stores a sequence of events: the submitted return, its review, its disposition and any later correction. A return may be admitted, sent for repair, retained as rejected, marked duplicate or superseded. Earlier entries remain available. If a review is corrected, the new decision must be carried through to the records that used the earlier one; retaining the old event makes that dependency inspectable.

The journal checks event order and hashes linking each event to its predecessor. A file lock serialises cooperating writers, and an append operation checks the current head before writing. This helps detect broken or altered records within the journal. The lock does not cover transactions over every source file; someone with control of the files and checking code can also change both. These are limits of the local history as evidence, even when its hashes agree.

After a maintainer adopts a change, the affected source and reviewed records are updated before their derived pages. The refresh script has an explicit order for builders and then checks whether another refresh would change the outputs. It can run independent read-only checks in parallel. The order is intended to keep a revised summary consistent with the record from which it was generated.

Publication checks and proof builds remain separate. The release script checks the configured publication requirements; it does not run Lean. The Lean continuous-integration workflow specifies the formal build. Both historical Lean source roots are subject to the same proof policy. That policy excludes `native_decide` in the relevant checks; this is a local trust choice, not a claim that the mechanism is generally unsound. Appendix A identifies the source files and commands for these different checks.

A successful return check makes a proposal ready for review. The maintainer must still assess the mathematics, sources, explanation and credit before accepting it. The workflow does not technically force a second independent mathematician to perform that assessment. A second agent using the same selected context can repeat the first agent's mistake, so the number of recorded reviews alone does not establish independent scrutiny.

An accepted revision may also suggest a change to the writing instructions. That is a separate decision: a useful edit in one paper may be poor advice for another. The project retains the proposed rule, the example, its limits and the integrating decision. Accepted changes revise documents or procedures used in later work. They do not train model weights.

5 Evidence from the prototype

5.1 The repository inventory

The retained inventory covers 8 problem collections, 1,903 local Lean modules, and 160 reviewed claims in eight statuses, with 19 recorded obligations. It lists 24 registered papers, including 8 short papers and 8 long records. At journal head `d3ad85269386`, there are 7 rounds, of which 2 are sealed, and 25 returns: 18 admitted and 7 unreviewed. These totals describe the supplied snapshot.

The paper-to-proof inventory contains 689 statement occurrences: 498 with exact support, 142 obtained by specialising stronger results, 23 depending on named inputs, and 26 without registered Lean support. Of the 640 occurrences in the first two categories, 616 have a recorded comparison and 24 are queued. The 663 occurrences with some Lean support include the named-input category. These categories count statement occurrences, not independent discoveries; repeated occurrences and overlapping supports prevent the totals from being read as a theorem census.

The broader interpretation index illustrates a different counting issue. It attaches descriptions to 139,818 of 152,386 theorem-like declarations, or 91.8%. Only 3,330 are classified as hav-

ing direct evidence; 136,488 use contextual interpretation, and 12,568 remain structural-only. The large coverage percentage must therefore not be presented as the proportion of declarations individually understood or reviewed.

5.2 A false claim that passed the checks

The historical publication-evidence record reports that nine of the ten deliberately false edits were rejected and one escaped. The edits were authored by the checker’s author; the original run logs were not retained, and the other nine edits were not rerun against the extended checklist. Each edit was applied separately, restoring the baseline between trials.

The escaped edit changed an account of an unresolved step from “does not supply” to “completes”. The publication checks accepted it. Table 1 places this failure beside the nine recorded rejections: several checks detected changed coordinates, labels or counts, while the false completion sentence left those objects intact.

Edit	Deliberate change	First recorded detector	Outcome
1	Conditional result labelled proved	Projection freshness	Rejected
2	Open-boundary clause deleted	Boundary wording	Rejected
3	Declaration coordinate moved	Source coordinates	Rejected
4	Paper source link retargeted	Source coordinates	Rejected
5	Claim assigned to two families	Registry structure	Rejected
6	Module count hand-edited	Projection freshness	Rejected
7	native_decide introduced	Local proof policy	Rejected
8	“does not supply” changed to “completes”	None	Escaped
9	Open-problem title mangled	Paper anchoring	Rejected
10	Orientation byte budget exceeded	Byte budget	Rejected

Table 1: The ten historical false edits, as reported in the retained evidence record. The checker author selected and authored the edits. The original run logs were not retained. The later follow-up tested only the baseline and edit 8.

A later check was added for the false clause. The saved follow-up record reports that the post-repair witness accepts the current README and rejects a test copy containing the false clause. Here “current” refers to the baseline in that follow-up record. The other nine edits were not tested against the extended checklist, so there is no post-repair ten-of-ten result. The study locates a coverage boundary. Its nine rejections do not estimate how reliable the checker is.

The trial used one corpus and the checker’s own selection of errors. There was no manual-review control or comparison with ordinary continuous integration. It shows that these specific checks rejected these recorded edits and missed a consequential change in meaning.

5.3 A local contribution handoff

On 23 September 2026, a maintainer-operated agent accepted one architecture handoff into a local integration history after focused checks of return routing and packaging. The same agent acted as contributor and reviewer. The record contains no live public pull request, independent mathematical review or outside-clone replay. It demonstrates a local path from a submitted artifact to a recorded integration decision, with those limits.

6 Relation to existing systems

Prove2Me is the closest comparison for collaborative mathematical work [31, Sections 3–4]. Its source-linked milestones pair a mathematical statement with a human-attested canonical formalisation. Its proof-sketch mechanism can compose verified subtheorems, with the parent verified only after its required subtheorems are verified. It also describes independent readback of formal statements and human review of the correspondence. These are substantive mechanisms for connecting a mathematical task to formal work. This repository’s coverage row has a narrower role: it lists supports for a paper occurrence and records their relation to that occurrence. Listing those supports does not perform Prove2Me’s proof composition. The additional passage record concerns explanations, including their historical and architectural claims.

Blueprints connect an informal mathematical argument to formalisation work. The Carleson project used its blueprint to initiate formalisation and revised it as that work exposed corrections and extensions [87, 35]. LeanArchitect attaches authored mathematical descriptions and metadata to Lean declarations, extracts dependencies and proof status, and exports material for a blueprint [74, Methods]. These approaches bring the mathematical explanation close to the formal source. The passage records described here also accommodate claims about a script’s behaviour, a historical test or prior attribution. They record why a source is cited without deriving the explanation from Lean.

The underlying techniques are established. Build systems track dependencies to decide which outputs must be rebuilt [48]. This repository uses fixed builder order and source hashes for a related purpose: identify derived records or explanations that need attention after an input changes. The extra relation note remains authored; a dependency check cannot determine whether the cited mathematics supports a new paraphrase. The repository applies dependency tracking to the continuing revision of research records and papers.

Souza and colleagues build an agent for querying workflow provenance, using a dynamically maintained schema to keep the language model’s context smaller than the raw execution history [94]. Abedu and colleagues construct a repository graph from users, commits, issues and files, then generate and execute graph queries to answer repository questions [95]. Both connect natural-language interaction to structured records. Here, the journal records entered research contributions and review decisions, while lookup tools return selected mathematical context.

Repository review itself is not new. The mathlib community describes contribution through pull requests and maintainer review, alongside public discussion [76, Section 7]. AGMAI’s responsible-release guidance asks for understandable mathematics, appropriate credit, disclosure of tools and process, and accessible artifacts [10]. The records here can help a reviewer inspect some of that information. They do not certify compliance with the whole guidance, supply an independent review, or turn formal validity into a judgement of mathematical importance.

7 Limits and the next evaluation

The main uncertainty is whether this organisation helps someone who did not build it. No independent reader study, adoption by another laboratory, discovery-rate comparison or measured reduction in review cost is reported. Nor has the system been compared with a well-maintained repository containing a good README and ordinary continuous integration. More records and more links may reduce reconstruction work, or may merely add another layer to maintain. The inventory cannot decide between those possibilities.

A useful first evaluation would give unfamiliar researchers fixed tasks: find the result answering a question, recover a necessary hypothesis, identify the relevant prior work, or diagnose a discrepancy between a paper and its record. Compare ordinary repository browsing with an

information-equivalent static brief and with the navigation tools. Holding the available information constant would help separate the benefit of selection from the benefit of the interface. Record wrong conclusions, omitted conditions, unresolved tasks and requests for help, as well as completion time.

The evaluation should include the cost of preparing records, keeping them current and repairing misleading entries. It should retain timeouts, failed attempts and reviewer disagreement, rather than reporting only successful tasks. Yuan and colleagues' study of open-hardware documentation offers a relevant test of the premise: unfamiliar students followed existing procedures and exposed ambiguities, missing practical detail and obsolete tooling [106]. That study concerns people using hardware documentation, not an automated check of explanatory prose. An equivalent outside-reader test of this repository remains to be done.

The public workflow also depends on explicit choices: someone must choose a worthwhile task, enter useful evidence and accept or reject its result. It does not provide an autonomous research scheduler. Volunteer computation, transfer to other sciences and a broader service for coordinating researchers remain proposals. No upstream mathlib contribution from this workflow is established by the retained records, and possible exposure of the source material during model training is unknown. These uncertainties should be resolved before attributing new discoveries or efficiency gains to the system.

8 Conclusion

This repository makes selected research records easier to inspect together: the question, attempted work, formal support, explanation and review decision. Its important design choice is to preserve the difference between them. The escaped completion claim shows why that distinction matters. A changed hash directs the reviewer to material that needs attention; a formal check establishes its specified formal statement. Someone still has to decide whether the paper says what the evidence permits. The next test is whether these records help an unfamiliar researcher make that decision accurately and at reasonable cost.

A Sources and checks

The [reading guide](#) leads from a problem to its paper, argument and formal-support references. Agents can use [the agent instructions](#) and the optional [proof workbench](#). The [contribution guide](#) and [credit policy](#) describe how to propose corrections. Reading these materials does not require a Lean installation.

The following source files answer different audit questions. `docs/claims.json` owns the selected reviewed assertions. `docs/publication_contract.json` lists publication requirements, and `docs/publication_evidence.json` preserves the historical false-edit record. `scripts/check_release.py` implements publication checks. For formal build entry points, inspect `lean/ErDOS249257.lean` and `.github/workflows/lean.yml`. Their presence or mention is not a record of a fresh execution.

From a complete public checkout, the document checks are:

```
python3 scripts/systems_paper_evidence.py
python3 scripts/build_systems_paper_counts.py
```

The first checks passage bindings and source hashes. The second checks the generated inventory against its records. A selected writing packet supports only the checks for which it includes the required files; selected Lean excerpts are not a complete proof environment. Proof replay and formal comparison require their own recorded revisions, dependencies and toolchains.

General repository links in this revision point to source snapshot 95cc5efc727974c05dc6842c349771627752c237. Individual proof and comparison records may name different revisions; their own pins govern replay. The earlier design accounts, *Cold Clone to Proof Receipt* and *Open-Source Mathematics Strategy*, retain historical detail and proposals. They are not additional evidence that those proposals now run in a public clone.

B The condition in the Problem 257 example

Let A be an infinite set of positive integers and P a finite nonempty set of primes. Write $h_P(a)$ for the largest divisor of a whose prime factors lie in P . The owning paper proves the following sufficient condition [2]:

$$W_{2,P}(A) := \sum_{a \in A} \frac{h_P(a)}{a(2^{h_P(a)} - 1)} < \infty \quad \Rightarrow \quad \sum_{a \in B} \frac{1}{b^a - 1} \notin \mathbb{Q} \quad (1)$$

for every infinite $B \subseteq A$ and every integer $b \geq 2$. The base-two condition implies the corresponding condition at every larger base. A condition imposed only at a fixed base gives the fixed-base conclusion; it should not silently be turned into an all-base assertion. The unrestricted Problem 257 remains open in this work.

Erdős proved an earlier irrationality criterion under pairwise coprimality and reciprocal summability, and stated that the coprimality restriction could be removed without supplying the details of that extension there [3]. The owning paper gives the extension argument. Duverney and Tachiya supply relevant selection and congruence ideas and the first averaging step [4]. The weighted proof adds an average over dyadic progression lengths; this supplies the factor $1/a$ needed in the displayed condition. Its proof, parameter choices and a separating example are in the Problem 257 paper, rather than repeated here [2, Section 2]. The separating example's calculation is an ordinary argument, not a separately registered Lean result.

Coverage row `res:weighted-support` names two overlapping supports in namespace `Erdo sProblems.Erdos257.PaperCompleteR8`: `divisibilityWeightedClaim` and `finitePrimeWeighted_fixedBase_hereditary`. Their source files are [WeightedReturn.lean](#) and [WeightedHereditaryClaim.lean](#). They support the stated conclusion; the row does not assert that the ordinary proof follows the Lean proofs line by line.

The weighted comparison entries are `PalomarCorpus/E257` and `E257bc`. The recorded runs are `35544127144` and `35624228171`, with comparison recorded as completed. The reciprocal criterion has its own declaration, `Erdos249257.irrational_erdosSupportSeries_of_summable_reciprocal`, and comparison entry `E257av`, recorded as compared in the second run. Palomar entries are prepared, not submitted. These statuses do not establish registry acceptance or historical attribution.

C Other scope and attribution records

Three other records illustrate why attribution and scope need their own review. They are not additional systems experiments. In Problem 269, the result concerns distinct running least common multiples; the original question counts repeated values with multiplicity. The paper credits Fan's earlier public two-prime result despite a reported independent derivation, and identifies the related Hecke–Mahler literature [28, 29, 26, 27]. Neither formalisation nor independent derivation removes that earlier credit.

For Problem 249, the paper gives explicit reductions and an integral basis for relations among totient subsequences [13]. Coons establishes non-regularity, while Martin's stronger separation result already implies the independence used for the basis. The direct argument uses the Chinese remainder theorem and Dirichlet's theorem through an argument in Yazdani, who credits

Shallit [14, 15, 16]. The explicit description, the inherited independence result and its formalisation are distinguishable contributions.

For Problem 1041, the degree-seven counterexample construction is due to the forum contributor ani [23, 21]. The recorded Lean results refute the precise Formal Conjectures statements, including the relevant path-image-length and total-variation formulations. Independent human review of their correspondence with the curve-length wording in the 1958 paper is not recorded [77, 24, 22]. The four declarations in namespace `Erdos1041.Counterexample` are `erdos1041_counterexample`, `erdos1041_counterexample_hausdorff`, `erdos1041_hausdorff_negation` and `erdos1041_hausdorff_answer_false`. Their coverage row records exact-or-stronger support, with Comparator pending. The other seven motivating targets remain unresolved in this work.

D Reference map

Section 6 compares the nearest systems by their mechanisms. The map below retains the wider references used in this paper and its two historical design accounts. A place in the map identifies a subject and a reason to consult a source; it does not mean that the source was evaluated against this implementation or that its method is implemented here. The bibliography is retained as a credit record.

Area	Reason for citation
Formal foundations	Lean and mathlib; library maintenance and contributor practice [11, 76, 92, 100]
Source and statement checks	Comparator, kernel replay, vacuity, linters and property tests [19, 20, 52, 43, 51]
Formal-to-prose tools	Blueprints, source documentation, translation and refactoring [35, 74, 37, 36, 61, 70]
Retrieval and interaction	Premises, proof states and exact declaration neighbourhoods [39, 88, 90, 91, 89, 93]
Proof automation	Premise selection and tactic search [49, 50, 59]
Library structure	Declaration networks and repository knowledge graphs [45, 46, 95]
Assumptions and reuse	Proof mining, generalisation, lemma mining and theory exploration [41, 42, 53, 54]
Growing proof knowledge	Library learning, evolving procedures and candidate statements [55, 56, 57, 58, 62, 83]
Research agents	Executable objectives, proof cost and evolving tasks [78, 79, 30, 25, 60]
Agent collaboration	Bounties, planner/worker state, retrieval and compiler feedback [80, 81, 82, 31, 44, 75]
Evaluation	Research budgets, proof engineering and documentation tasks [84, 71, 72, 73, 106, 102, 103]
Validity and intended meaning	Research case studies, interactive proving and public accounting [18, 32, 40, 33]
Public infrastructure	Problem sources, formal conjectures, verification and collaboration [1, 77, 101, 87, 99, 97, 98, 104, 105]
Writing and maintenance	Mathematical exposition, build dependencies and provenance [85, 69, 48, 47, 94]
Understanding and reuse	Explanation, mathematical understanding, reuse and research possibilities [5, 6, 9, 86, 7, 63, 96]
Different community views	Declarations, dissent, benchmarks, research infrastructure and pace [8, 64, 65, 66, 34, 67, 68]
Mathematical antecedents	Weighted support, totient ranks and arithmetic separation [2, 3, 4, 13, 14, 15, 16]
Other mathematical records	Running LCM, lemniscates and cross-problem connections [28, 29, 26, 27, 12, 38, 22, 21, 23, 24, 17]

Figure 2: Reading map for this paper and its two historical predecessors. The groups record uses of sources, not a performance ranking or a list of implemented components. Appendices B and C distinguish mathematical antecedents, refinements and independent rediscovery.

References

- [1] T. F. Bloom, *Erdős problems*, erdosproblems.com, accessed September 2026.
- [2] W. Cook, *Weighted and Covered Supports for Mersenne Subseries*, short paper on Erdős Problem 257, September 2026, [PDF](#).
- [3] P. Erdős, *On the irrationality of certain series*, Math. Student 36 (1968), 222–226 (issued 1969), [scan](#).
- [4] D. Duverney and Y. Tachiya, *Refinement of the Chowla–Erdős method and linear independence of certain Lambert series*, Forum Math. 31 (2019), 1557–1566, [DOI](#).
- [5] B. Kra, *Deep theorems were scarce and difficult and so became an effective mechanism to identify deep thought. AI has broken this system*, guest post on *What’s new*, 13 September 2026, [blog post](#).
- [6] H. Cohn, *The technical debt of AI-generated mathematics*, guest post on *What’s new*, 15 September 2026, [blog post](#).
- [7] T. Tao, thread on mining open problems, Mathstodon, 8 September 2026, [thread](#).
- [8] Twenty-five Fields Medallists, *A severe misalignment of AI in mathematics*, declaration posted on *What’s new*, 11 September 2026, [blog post](#).
- [9] G. Sanderson, *If math is more than proof, we need to better celebrate the rest of it*, guest post on *What’s new*, 18 September 2026, [blog post](#).
- [10] Advisory Group on Mathematics and Artificial Intelligence at IAS, *Responsible Release of AI-Generated Mathematics*, 29 September 2026, [recommendations](#), [PDF](#).
- [11] L. de Moura and S. Ullrich, *The Lean 4 Theorem Prover and Programming Language*, in *Automated Deduction, CADE 28*, Lecture Notes in Computer Science 12699, 2021, pp. 625–635, [DOI](#).
- [12] P. Erdős and R. L. Graham, *Old and New Problems and Results in Combinatorial Number Theory*, Monographies de L’Enseignement Mathématique 28, 1980, p. 61.
- [13] W. Cook, *Integral Relations among Totient Sections*, short paper on Erdős Problem 249, revised 30 September 2026, [PDF](#).
- [14] M. Coons, *(Non)Automaticity of number theoretic functions*, J. Théor. Nombres Bordeaux 22 (2010), 339–352, [DOI](#), Theorem 3.2.
- [15] G. Martin, *Simultaneous inequalities among values of the Euler phi-function*, 2006, [arXiv:math/0603053v1](#), Theorem 1.
- [16] S. Yazdani, *Multiplicative functions and k-automatic sequences*, J. Théor. Nombres Bordeaux 13 (2001), 651–658, [Numdam](#).
- [17] W. Cook, *Reading Eight Erdős Problems Together*, synthesis paper, September 2026, [source](#), [including the periodic-chain theorem](#).
- [18] T. Feng, T. Trinh, G. Bingham, et al., *Semi-Autonomous Mathematics Discovery with Gemini: A Case Study on the Erdős Problems*, 2026, [arXiv](#).
- [19] Lean FRO, *Comparator*, 2025, [GitHub](#).

- [20] ammkrn, *nanoda_lib*, an independent type checker for Lean 4, [GitHub](#).
- [21] T. F. Bloom, *Erdős Problem #1041*, [erdosproblems.com](#), accessed September 2026.
- [22] P. Erdős, F. Herzog and G. Piranian, *Metric properties of polynomials*, J. Analyse Math. 6 (1958), 125–148, [DOI](#).
- [23] ani, post in the Problem 1041 discussion thread, 7 September 2026, [erdosproblems.com forum](#).
- [24] Formal Conjectures, pull request 6505, *Erdős 1041: mark solved with answer(False) and link a formal proof*, merged 23 September 2026, [GitHub](#).
- [25] T. Zheng, X. Wu, Z. Zhang, et al., *Dream-RSI: Recursive Self-Improvement through Evolving Worlds*, 2026, [arXiv:2609.14858](#).
- [26] J. H. Loxton and A. J. van der Poorten, *Arithmetic properties of certain functions in several variables III*, Bull. Austral. Math. Soc. **16** (1977), 15–47.
- [27] Y. Bugeaud and M. Laurent, *Transcendence and continued fraction expansion of values of Hecke–Mahler series*, Acta Arith. **209** (2023), 59–90.
- [28] W. Cook, *Distinct running least common multiples*, short paper on Erdős Problem 269, September 2026, [PDF](#).
- [29] S. Fan, comment on Erdős Problem #269, [erdosproblems.com forum](#), 26 June 2026, [forum post](#).
- [30] N. Patel, A. Rammal, A. Hayat, R. Munos, and J. Kempe, *Learning to Discover Interesting Mathematics*, [arXiv:2609.28603v1](#), 2026, [arXiv:2609.28603v1](#).
- [31] S. Chen, K. Marwaha, X. Lu, H. Yuen, and T. Peng, *Prove2Me: An Open Collaborative Platform for Scaling Math Formalization*, 2026, [arXiv](#).
- [32] T. Feng, T. H. Trinh, G. Bingham, et al., *Towards Autonomous Mathematics Research*, 2026, [arXiv](#).
- [33] T. Tao, *AI contributions to Erdős problems*, [GitHub](#), accessed September 2026.
- [34] T. Ringer, *Becoming a benchmark*, guest post on *What’s new*, 17 September 2026, [blog post](#).
- [35] P. Massot, *leanblueprint*, plasTeX plugin for Lean formalisation blueprints, 2020, [software repository](#).
- [36] Y. Zhang, Y. Sun, T. Suzuki, J. D. Lee, and F. Liu, *LeanMarathon: Toward Reliable AI Co-Mathematicians through Long-Horizon Lean Autoformalization*, 2026, [arXiv:2606.05400](#).
- [37] S. Hattori, T. Matsuzaki, and M. Fujiwara, *Natural Language Translation of Formal Proofs through Informalization of Proof Steps and Recursive Summarization along Proof Structure*, in *Proceedings of the 18th International Natural Language Generation Conference*, 2025, pp. 376–389, [ACL Anthology](#).
- [38] S. Akiyama and H. Kaneko, *Multiplicative analogue of Markoff–Lagrange spectrum and Pisot numbers*, Adv. Math. **380** (2021), 107547, p. 3, reporting the theorem of A. Dubickas, J. Number Theory **117** (2006), 222–239, [arXiv:1911.06170v6](#).
- [39] K. Yang et al., *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models*, NeurIPS 2023, Datasets and Benchmarks Track, [arXiv:2306.15626](#).

- [40] C. Li, Z. Lai, D. An, J. Hu, and Z. Wen, *Advancing Mathematical Research via Human-AI Interactive Theorem Proving*, 2025, [arXiv:2512.09443v2](#).
- [41] U. Kohlenbach, *Applied Proof Theory: Proof Interpretations and their Use in Mathematics*, Springer Monographs in Mathematics, Springer, Berlin, 2008.
- [42] A. Gandhi, A. R. Tadipatri, and T. Gowers, *Automatically Generalizing Proofs and Statements*, in *16th International Conference on Interactive Theorem Proving*, LIPIcs 352, 2025, pp. 12:1–12:18, [DOI](#).
- [43] Lean community, *Batteries*, the unusedArguments linter in Batteries/Tactic/Lint/Misc .lean, [GitHub](#), accessed September 2026.
- [44] J. Xin, A. Schneidman, C. Cummins, K. Ram, S. Ganesh, and J. Limperg, *AXLE: A Cloud Infrastructure for Lean 4 Theorem Proving Utilities*, 2026, [arXiv:2606.26442](#).
- [45] X. Li, N. Peng, S. Severini, and P. Shafto, *The Network Structure of Mathlib*, 2026, [arXiv:2604.24797](#).
- [46] F. Huch, *Structure in Theorem Proving: Analyzing and Improving the Isabelle Archive of Formal Proofs*, extended abstract, 2022, [arXiv:2209.13305](#).
- [47] J. de Kleer, *An assumption-based TMS*, Artificial Intelligence 28 (1986), 127–162, [DOI](#).
- [48] A. Mokhov, N. Mitchell, and S. Peyton Jones, *Build Systems à la Carte*, Proc. ACM Program. Lang. 2 (ICFP), 2018, article 79, [DOI](#).
- [49] T. Zhu, J. Clune, J. Avigad, A. Q. Jiang, and S. Welleck, *Premise Selection for a Lean Hammer*, 2025, revised 2026, [arXiv:2506.07477](#).
- [50] J. Limperg and A. H. From, *Aesop: White-Box Best-First Proof Search for Lean*, in *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs*, 2023, pp. 253–266, [DOI](#).
- [51] Lean community, *Plausible*, a property testing framework for Lean 4, [GitHub](#), accessed September 2026.
- [52] H. Chockler, A. Gurfinkel, and O. Strichman, *Beyond Vacuity: Towards the Strongest Passing Formula*, Form. Methods Syst. Des. 43 (2013), 552–571, [DOI](#).
- [53] C. Kaliszyk and J. Urban, *Lemma Mining over HOL Light*, LPAR-19, LNCS 8312, 2013, pp. 503–517, [DOI](#).
- [54] M. Johansson, D. Rosén, N. Smallbone, and K. Claessen, *Hipster: Integrating Theory Exploration in a Proof Assistant*, CISM 2014, LNCS 8543, pp. 108–122, [DOI](#).
- [55] K. Ellis et al., *DreamCoder: Growing Generalizable, Interpretable Knowledge with Wake-Sleep Bayesian Program Learning*, 2020, [arXiv:2006.08381](#).
- [56] H. Wang, H. Xin, et al., *LEGO-Prover: Neural Theorem Proving with Growing Libraries*, ICLR 2024, [arXiv:2310.00656](#).
- [57] Y. Zhang, J. Sun, et al., *DreamProver: Evolving Transferable Lemma Libraries via a Wake-Sleep Theorem-Proving Agent*, 2026, [arXiv:2604.26311](#).
- [58] W. Ye, Z. Guan, et al., *ProofEvolve: Neuro-Symbolic Evolution for Formal Automated Theorem Proving*, 2026, [arXiv:2608.26334](#).

- [59] S. Böhme and T. Nipkow, *Sledgehammer: Judgement Day*, IJCAR 2010, LNCS 6173, pp. 107–121, [DOI](#).
- [60] I. Berlot-Attwell, T. Sesterhenn, F. Rudzicz, and X. Si, *Is This LLM Library Learning? Evaluation Must Account For Compute and Behaviour*, EACL 2026, pp. 3534–3568, [DOI](#).
- [61] Y. Fu, P. Liu, Z. Wang, and K. Yuan, *Proof-Refactor: Refactoring Generated Formal Proofs into Modular Artifacts*, 2026, [arXiv:2606.03743](#).
- [62] N. Onda et al., *LeanConjecturer: Automatic Generation of Mathematical Conjectures for Theorem Proving*, 2025, [arXiv:2506.22005](#).
- [63] D. Litt, *A beginning for mathematics*, 13 September 2026, [blog post](#).
- [64] T. Chu, *The AI dissenter viewpoint*, *Proofs and Prompts*, 9 August 2026, [blog post](#).
- [65] B. Green and forty-one other Fellows and Foreign Members of the Royal Society, *Open letter to Sir Paul Nurse, President of the Royal Society*, *Proofs and Prompts*, 17 September 2026, [blog post](#).
- [66] W. T. Gowers, *Why I didn't sign the Fields medallists' letter*, 17 September 2026, [blog post](#).
- [67] D. Koukoulopoulos, *A CERN for AI-assisted science?*, guest post on *What's new*, 17 September 2026, [blog post](#).
- [68] B. Antieau, *Fast math/slow math*, 15 September 2026, [blog post](#).
- [69] D. E. Knuth, T. Larrabee and P. M. Roberts. *Mathematical Writing*. Stanford report CS1193, based on the autumn 1987 course, Section 1.
- [70] Dayu Yang et al., *DocAgent: A Multi-Agent System for Automated Code Documentation Generation*, [arXiv:2504.08725v3](#), 2025, Sections 2–3.
- [71] Xinchun Wang, Ruida Hu, Cuiyun Gao, Pengfei Gao, and Chao Peng. *Evaluating Repository-level Software Documentation via Question Answering and Feature-Driven Development*. [arXiv:2604.06793v1](#), 2026.
- [72] Huajian Xin, Luming Li, Xiaoran Jin, Jacques Fleuriot, and Wenda Li. *APE-Bench: Evaluating Automated Proof Engineering for Formal Math Libraries*. [arXiv:2504.19110v3](#), 2026.
- [73] Auguste Poiroux, Gail Weiss, Viktor Kunčák, and Antoine Bosselut. *Reliable Evaluation and Benchmarks for Statement Autoformalization*. [arXiv:2406.07222v3](#), 2025.
- [74] T. Zhu, P. Monticone, J. Avigad and S. Welleck. *LeanArchitect: Automating Blueprint Generation for Humans and AI*. [arXiv:2601.22554v1](#), 2026, Methods and Case Studies.
- [75] M. Bolan et al. *The Equational Theories Project: Advancing Collaborative Mathematical Research at Scale*. [arXiv:2512.07087v2](#), 2025, Project Management and Data Management.
- [76] The mathlib Community. *The Lean Mathematical Library*. CPP 2020. DOI: [10.1145/3372885.3373824](#).
- [77] Google DeepMind. *Formal Conjectures*. [Repository README](#), consulted 29 September 2026; descriptive context only.
- [78] A. Novikov et al. *AlphaEvolve: A coding agent for scientific and algorithmic discovery*. [arXiv:2506.13131v1](#), 2025, Task Specification and Evolution.

- [79] B. Georgiev, J. Gómez-Serrano, T. Tao and A. Z. Wagner. *Mathematical exploration and discovery at scale*. [arXiv:2511.02864v3](#), 2025.
- [80] C. E. Brown, C. Kaliszyk and J. Urban. *Agent Hunt: Bounty Based Collaborative Autoformalization With LLM Agents*. [arXiv:2603.06737v1](#), 2026.
- [81] M. Kripner and M. Straka. *OpenProver: Agentic and Interactive Theorem Proving with Lean 4*. [arXiv:2607.09217v1](#), 2026, State and Memory Management.
- [82] D. Ma et al. *OProver: A Unified Framework for Agentic Formal Theorem Proving*. [arXiv:2605.17283v1](#), 2026, Introduction.
- [83] Z. Yang et al. *CircuitProver: Agentic Lean 4 Theorem Proving with Reusable Circuit Proof Library for Hardware Verification*. [arXiv:2607.27259v1](#), 2026.
- [84] H. Wijk et al. *RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts*. [arXiv:2411.15114v1](#), 2024; official abstract used here.
- [85] P. R. Halmos. *How to Write Mathematics*. L'Enseignement Mathématique 16 (1970), 123–152; sections on organising a mathematical presentation.
- [86] Various authors. *Writings on AI and Mathematics*. Supplied anthology, AI_ESSAYS_SOURCE.pdf, snapshot 22 September 2026; Kra, pp. 31–33; Riehl, pp. 39–40; Cohn, pp. 41–44; Gowers, pp. 55–60; Sanderson, pp. 73–77. Riehl’s essay is also available as [Why I do mathematical research](#).
- [87] L. Becker et al., *A Blueprint for the Formalization of Carleson’s Theorem on Convergence of Fourier Series*, 2025, [arXiv:2405.06423](#).
- [88] L. Aniva, C. Sun, B. Miranda, C. Barrett, and S. Koyejo, *Pantograph: A Machine-to-Machine Interaction Interface for Advanced Theorem Proving, High Level Reasoning, and Data Extraction in Lean 4*, in *Tools and Algorithms for the Construction and Analysis of Systems*, 2025, pp. 116–137, [DOI](#).
- [89] S. Kurgan et al., *TheoremGraph: Bridging Formal and Informal Mathematics*, 2026, [arXiv:2606.25363](#).
- [90] J. Asher, *LeanExplore: A Search Engine for Lean 4 Declarations*, 2025, [arXiv:2506.11085](#).
- [91] G. Gao et al., *LeanSearch v2: Global Premise Retrieval for Lean 4 Theorem Proving*, 2026, [arXiv:2605.13137](#).
- [92] A. Baanen, M. R. Ballard, J. Commelin, B. Gin-ge Chen, M. Rothgang, and D. Testa, *Growing Mathlib: Maintenance of a Large Scale Mathematical Library*, in *Intelligent Computer Mathematics*, 2025, [arXiv:2508.21593](#).
- [93] B. Yanahama and A. Sannai, *Lean Atlas: An Integrated Proof Environment for Scalable Human–AI Collaborative Formalization*, 2026, [arXiv:2604.16347](#).
- [94] R. Souza, T. Poteet, B. Etz, D. Rosendo, A. Gueroudji, W. Shin, P. Balaprakash, and R. Ferreira da Silva, *LLM Agents for Interactive Workflow Provenance: Reference Architecture and Evaluation Methodology*, WORKS at the ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis, 2025, [DOI](#).
- [95] S. Abedu, S. Khatoonabadi, and E. Shihab, *Synergizing LLMs and Knowledge Graphs: A Novel Approach to Software Repository-Related Question Answering*, 2024, revised 2025, [arXiv:2412.03815](#).

- [96] T. Tao, *Mathematics in the age of AI*, 2026, [arXiv:2608.16753](#).
- [97] D. P. Anderson, *BOINC: A Platform for Volunteer Computing*, *Journal of Grid Computing* 18 (2020), 99–122, [DOI](#).
- [98] Great Internet Mersenne Prime Search, *GIMPS*, project documentation and discovery-credit record, [mersenne.org](#), accessed August 2026.
- [99] Polymath Project, *General polymath rules*, [project rules](#), accessed August 2026.
- [100] Lean community, *Contributing to mathlib*, [contributor guide](#), accessed August 2026.
- [101] Palomar Registry, *About Palomar and Contribution policy*, [registry documentation](#) and [submission standard](#), accessed August 2026.
- [102] J. Henkel, *The Mathematician’s Assistant: Integrating AI into Research Practice*, 2025, [arXiv:2508.20236](#).
- [103] J. Dekoninck et al., *The Open Proof Corpus: A Large-Scale Study of LLM-Generated Mathematical Proofs*, 2025, [arXiv:2506.21621](#).
- [104] NISO, *CRedit: Contributor Roles Taxonomy*, [role definitions](#), accessed August 2026.
- [105] GitHub, *Preventing pwn requests*, GitHub Actions security guidance, [documentation](#), accessed August 2026.
- [106] M. Yuan et al., *End-to-End Testing of Open-Source Hardware Documentation Developed in Large Collaborations*, 2023, [arXiv:2309.05942](#).